
Particije i shuffling

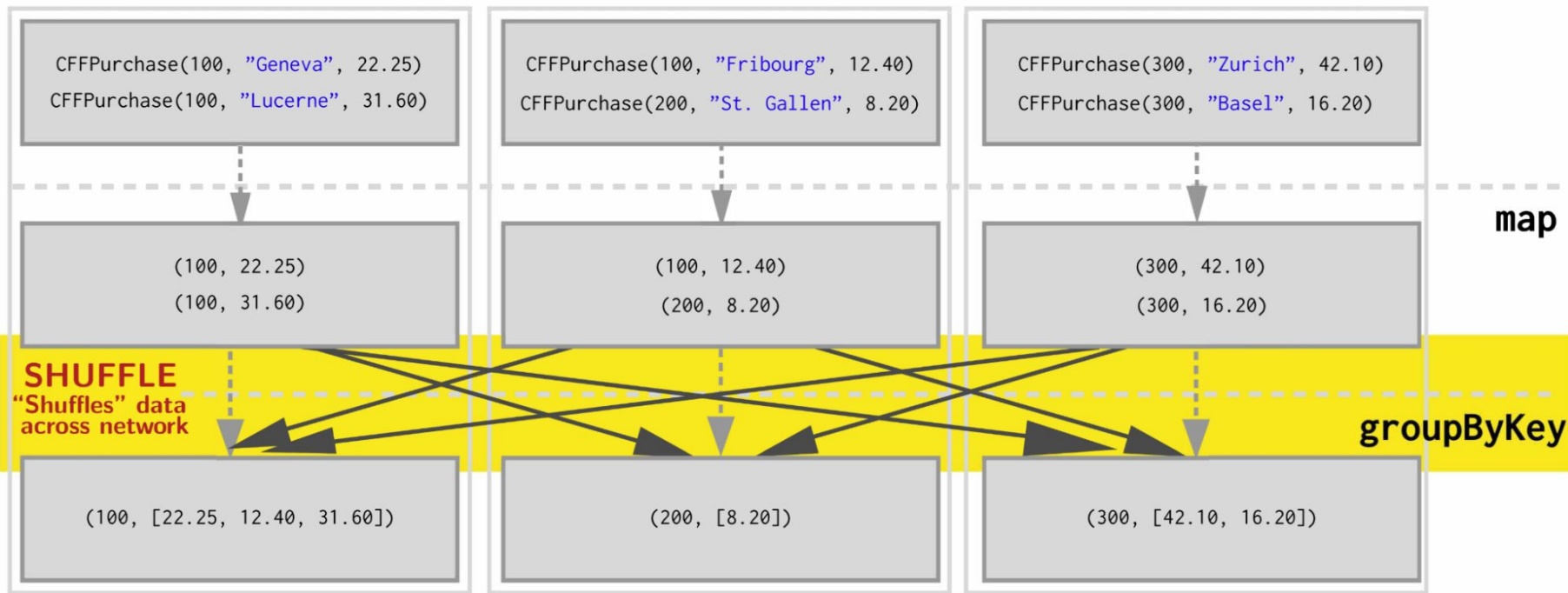
— Oporavak od greške, wide and narrow dependencies —

Početni primjer

- Word count sa prve sedmice
- Zašto varijanta sa groupByKey radi puno sporije?
- Zašto ima veću razliku u vremenu ako koristimo take(10) u odnosu na varijantu koja radi reduceByKey?
- Da li ikad trebamo koristiti groupByKey?

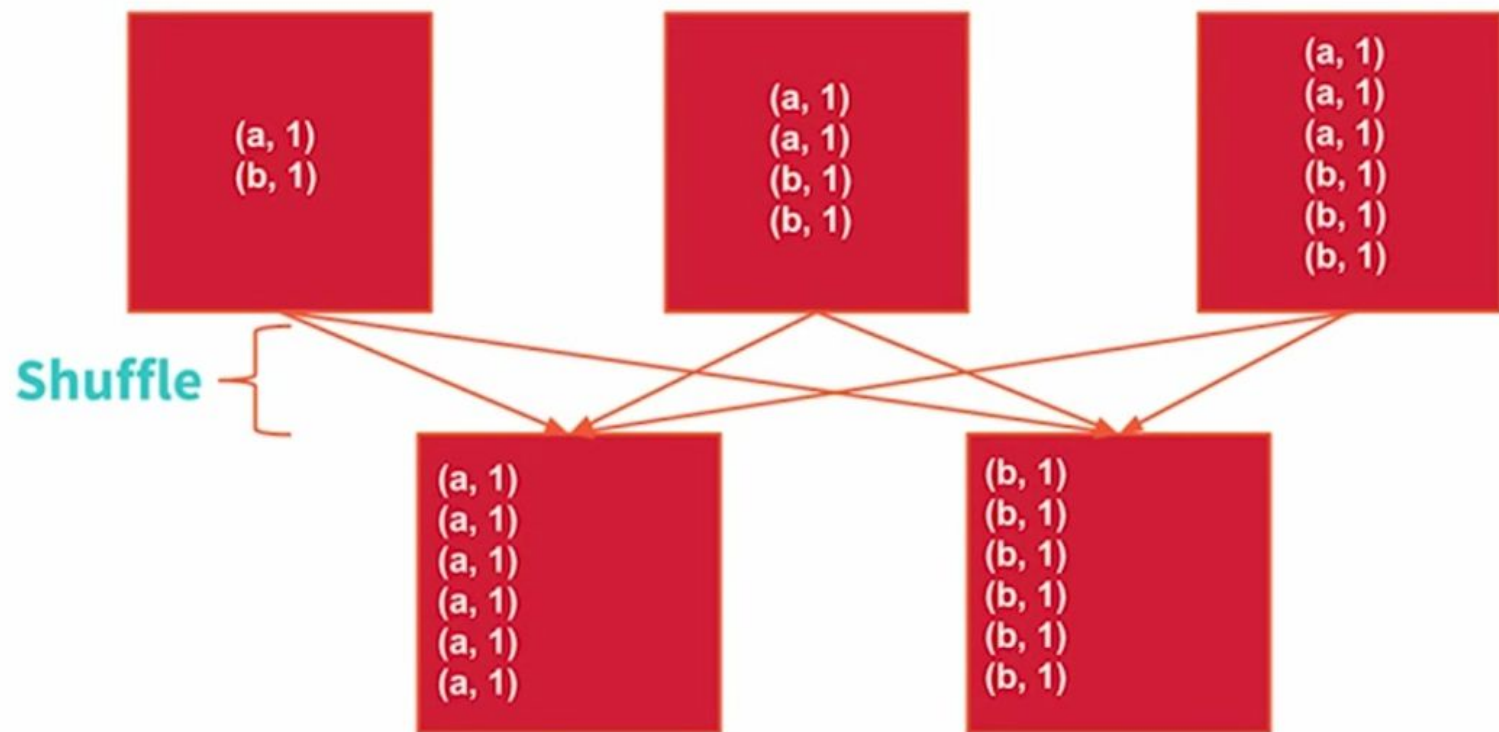
Shuffling

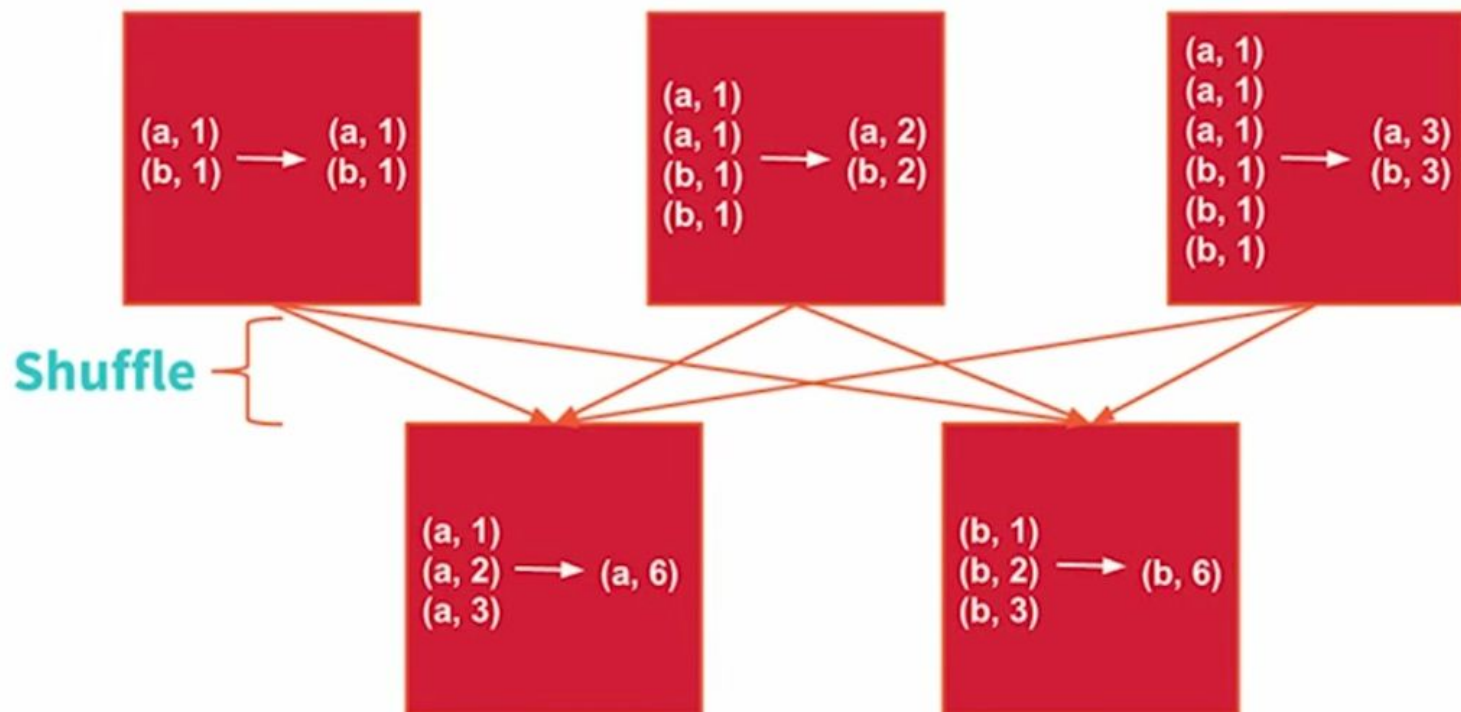
- Najskuplja operacija u distribuiranom programiranju - komunikacija preko mreže
- Framework je automatski rješava - da li možemo uticati?
- Izbor operacija i funkcija koje koristimo najbitniji!
- Kako funkcioniše groupByKey i zašto ga izbjegavamo?
- Rezultate groupByKey - ključ i lista vrijednosti, gdje se sve vrijednosti nalaze na istom node-u
- Kako izgleda na clusteru?



Parcijalna redukcija i shuffling

- Ako hoćemo konačan rezultat nemoguće je u potpunosti izbjeći komunikaciju preko mreže
- Minimizacija komunikacije cilj
- Kako funkcionirše reduceByKey?
 - Na svakom node-u ćemo prvo da uradimo redukciju koliko je moguće - sa podacima koji su dostupni
 - shuffling agregiranih podataka sa svakog node-a kao u groupByKey
 - finalna redukcija već djelimično agregiranih podataka
- Osnovna ideja - želimo što više da agregiramo podatke prije komunikacije preko mreže





Kako Spark inicijalno dijeli podatke?

- Partitionisanje - inicijalna podjela podataka po node-ovima na osnovu funkcije za particije
- Podaci jedne particije se uvijek nalaze na istoj mašini
- Svaka mašina ima jednu ili više particija - konfigurabilno - podrazumijevanu odgovara broju core-ova na svim executor node-ovima
- Tri tipa funkcije za particije:
 - hash (default)
 - range
 - custom

Hash i range partitioner

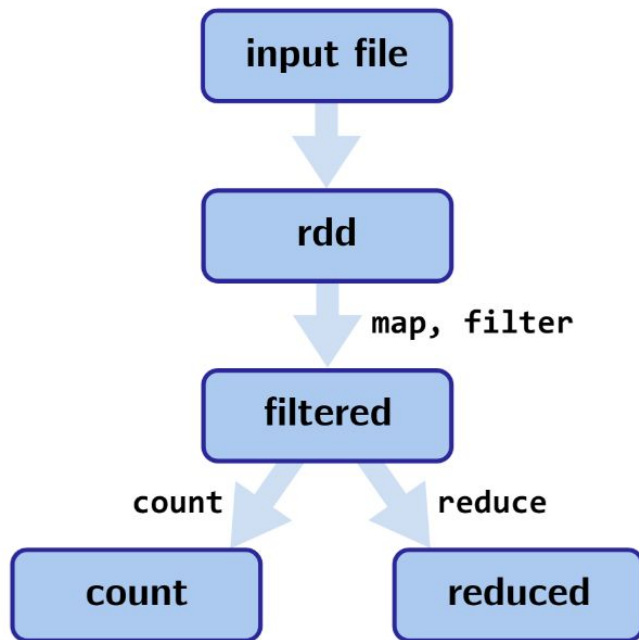
- Hash funkcija: $\text{partition} = \text{hash}(\text{key}) \% \text{num_partitions}$
- Range funkcija: partition = index of range containing key k
- Custom partition funkcije moguće
- Koristiti samo kada dobro razumijemo naše podatke
- Paziti na transformacije podataka ako koristimo naš partitioner
 - ukoliko koristimo map - gubimo naše particije
 - savjet da se koristi mapValues ukoliko je moguće

Oporavak od greške

- Jedno od najbitnijih osobina koda - kako reaguje na greške
- Dodatni problem u odnosu na standardne slučajeve - kod se jako dugo izvršava zbog velikih količina podataka
- Ne želimo da izgubimo sve što smo uradili
- Replay samo dijela operacija gdje se desila greška - minimizacija posla
- Spark generiše DAG (directed acyclic graph) za svoje operacije koje treba da odradi u okviru RDD operacija - lineage graph

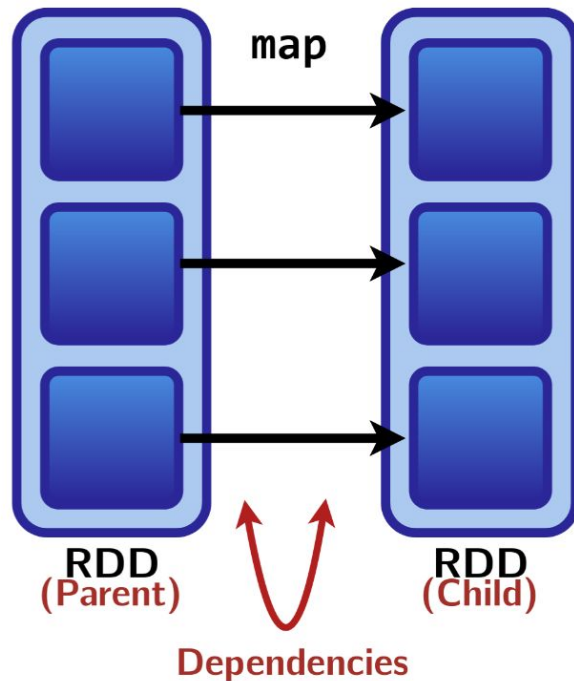
```
val rdd = sc.textFile(...)
val filtered = rdd.map(...)
                  .filter(...)
                  .persist()

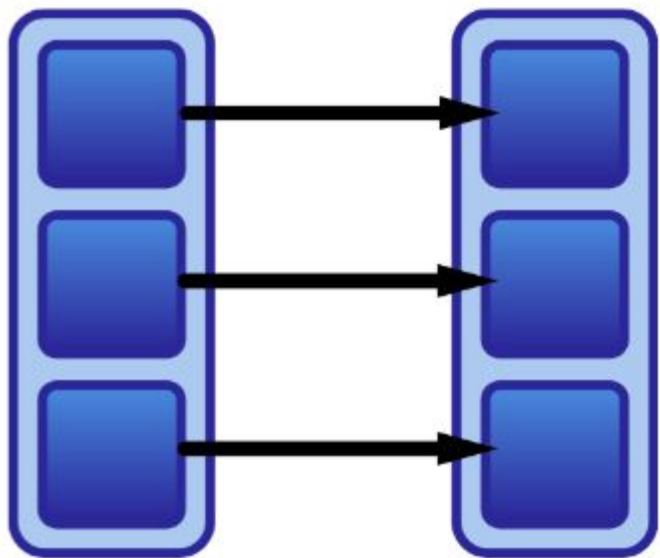
val count = filtered.count()
val reduced = filtered.reduce(...)
```



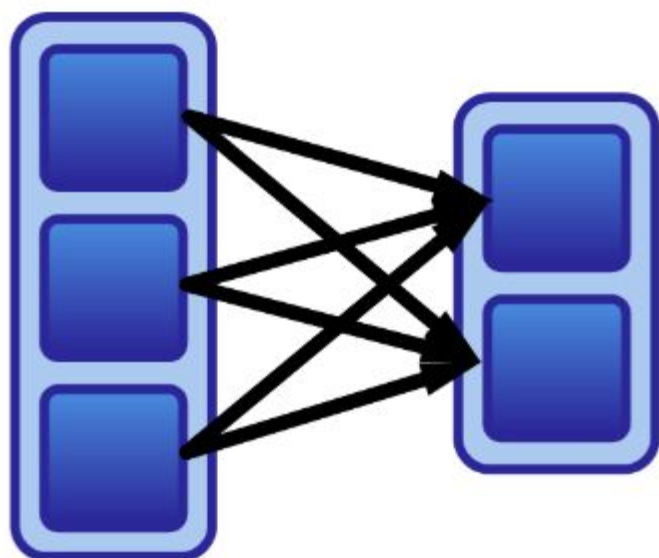
Uske i široke zavisnosti

- Svaka operacija u sparku uvodi novu zavisnost - trenutna operacija zavisi od prethodnih operacija
- Optimizacija ovih zavisnosti bitna
- Shuffling ponovo glavni problem
- Više shufflinga - sporiji oporavak od greške
- Zavisnost jedne particije od više drugih





map, filter



groupByKey

Transformations with narrow dependencies:

map

mapValues

flatMap

filter

mapPartitions

mapPartitionsWithIndex

Transformations with wide dependencies:

(might cause a shuffle)

cogroup

groupWith

join

leftOuterJoin

rightOuterJoin

groupByKey

reduceByKey

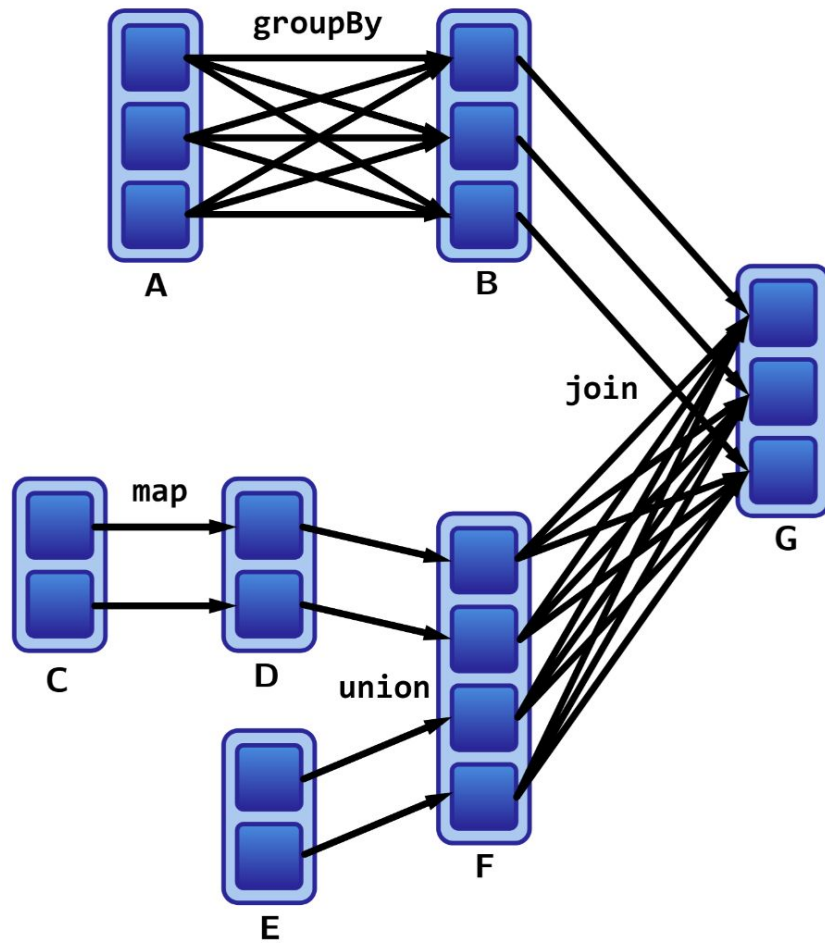
combineByKey

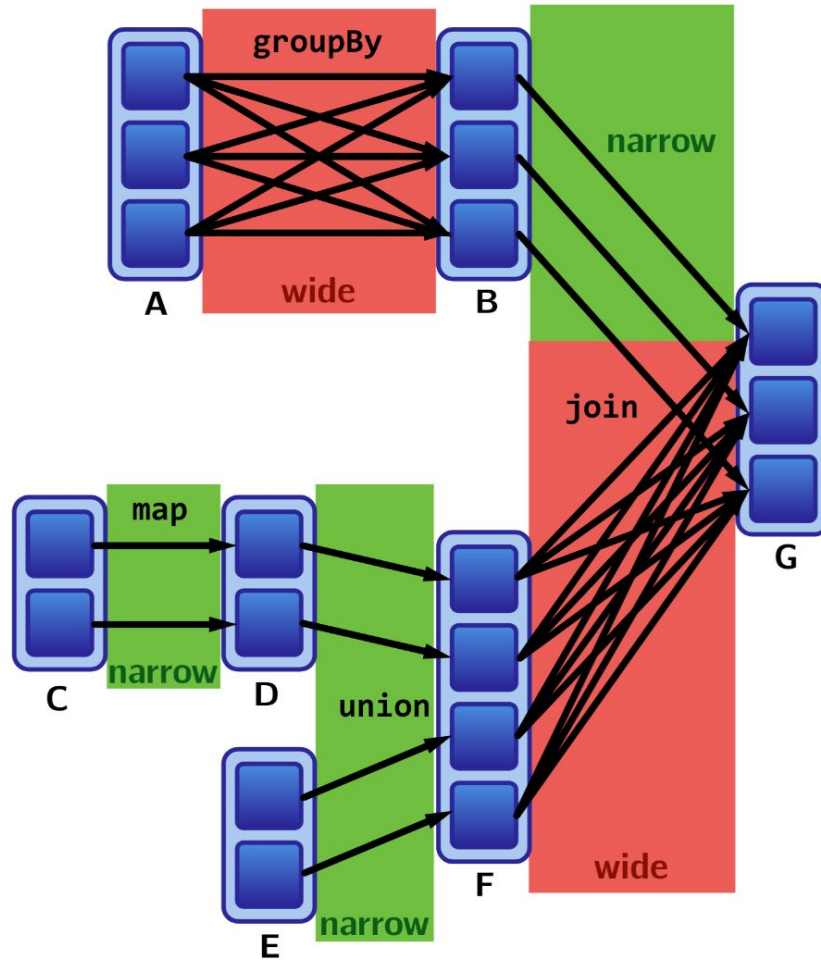
distinct

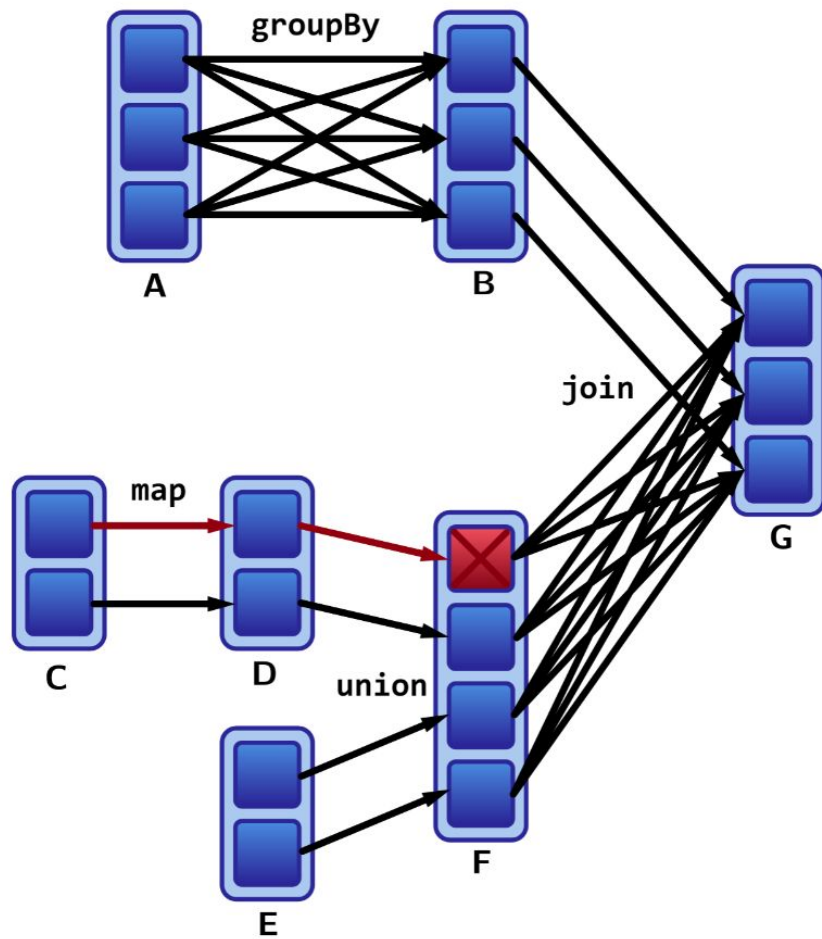
intersection

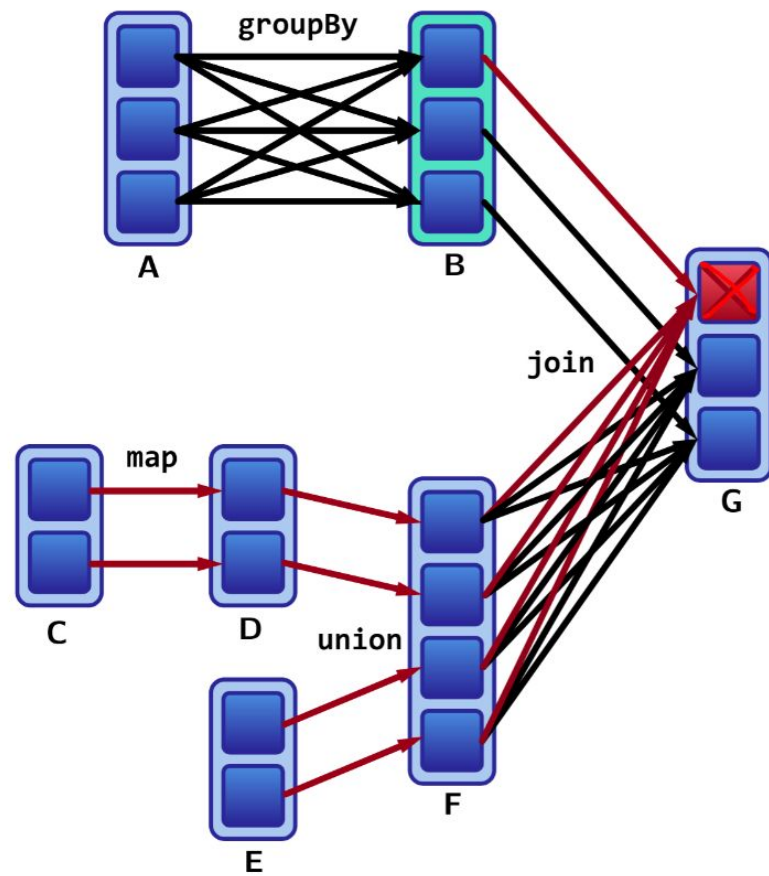
repartition

coalesce









Kako minimizovati probleme?

- Paziti gdje i kad koristimo metode koje koriste shuffle
- dependencies i toString metode nad RDD - omogućavaju da vizualizujemo i vidimo graf izračunavanja i zavisnosti u kodu

Narrow dependency objects:

- ▶ OneToOneDependency
- ▶ PruneDependency
- ▶ RangeDependency

Wide dependency objects:

- ▶ ShuffleDependency